数据结构 · 排序专题 · 考点精讲与真题整合

408 各科大题知识点强化课 | 课程第 4 讲（分 P6） | 时长 3 小时 28 分 44 秒 | 数据结构板块最后一讲

本讲范围：① 排序总观（考情 + 学每个排序的六个角度）；② 插入类：直接插入、折半插入、希尔；③ 交换类：冒泡、快速排序；④ 选择类：简单选择、**堆排序**（堆的定义 / 建立 / 插入 / 删除四个动作，本讲最重的一块）；⑤ **堆的应用**（今年大纲新增，第七题：用大根堆找最小的 10 个数）；⑥ 基数排序；⑦ 二路归并；⑧ **外部排序**（磁盘块与缓冲区、置换选择排序、败者树、最佳归并树）。

读法建议：§3–§4 是主体，每个排序都配了同一序列 `[6,1,5,9,8,4,7]` 的逐趟推演 + 比较/移动次数 + 时空复杂度 + 稳定性 + 存储结构；**§6.1 的六问模板**与 **§7 速查卡**最该背；§5 是外部排序，§附 放了算式独立验算清单与引语回对清单。

一句话：「**排序这边其实大题考的不多，但是选择题考得很琐碎**」；「**难点就是你要把它整理好并且记住**」——本讲值钱的是**同一串数据在每个排序里的逐趟过程**，以及**堆的三个动作各自的比较次数口径**。

目录

1 考情分析：排序在 408 里的分量

2 六个角度 + 三个前置概念

3 六种内排序逐个推演（同一序列）

4 堆三动作、堆排序、堆应用、基数与归并

5 外部排序（置换选择、败者树、最佳归并树）

6 解题方法论与易错点清单

7 考场速查卡

附 来源、边界与两步自查

ASR 校正说明（读引用前先看这一段）

转写稿由本地 Whisper 生成，本讲**全程无板书**（所有序列、堆、归并段都是口述 + 现场手画，画面看不到）。引用处已统一校正，主要替换：**枢轴**（输走 / 数轴 / 书轴 / 出轴 / 输轴）→ 枢轴；**趟数**（烫素 / 炭腐 / 胖数 / 汤数 / 烫熟）→ 趟数；**比较次数 / 移动次数**（比较制度 / 比较自主 / 比较之处 / 移动制度）→ 比较次数 / 移动次数；**归并**（规定 / 规避 / 规遍 / 归兵 / 固并）→ 归并；**磁盘块**（吃饭块 / 赤寒框 / 尺寒框 / 池瓣框 / 子凡话 / 池帆阔 / 十三块）→ 磁盘块；**堆**（对 / 队）→ 堆、**大根堆**（大更对）→ 大根堆；**完全二叉树**（完全二叉数 / 完全好插书 / 二差数）→ 完全二叉树；**败者树**（败者数 / 败则数）→ 败者树、**冠军节点**（冠军减点）→ 冠军节点；**置换选择排序**（自换选择排序）→ 置换选择排序；**数组**（宿主 / 竖竹 / 速度 / 数竹）→ 数组；**真题**（整体 / 真体 / 争题）→ 真题；**课下**（刻下 / 客家）→ 课下；**折半**（择办 / 择拌）→ 折半；**基数排序**（技术排序 / 基础排序）→ 基数排序；**有序**（有趣 / 有续 / 游戏）→ 有序。

数字类与拼不回来的地方不在本表展开，逐条列在 §附「引语回对」清单里。

1 考情分析：排序在 408 里的分量

「**排序这边其实大题考的不多**，但是选择题的话其实**考得很琐碎**。」「**对于考试来说，数据结构就 11 个选择题、两个大题**。」「他（一般）不会说一年（里）这个排序可能就只占一个选择题或者一个大题。」「他可能想着考得更全面的，就可能让你考**对比**：一个选择题提到好几个排序让你去选，**这样考得更全面**。」

> 「因为排序这块只靠选择，基本没有大题——**就一个像刚才讲的堆排序还有外部排序考过大题，而且都是比较冷门的，并不算特别常规的。**」「当然这边也有一道排序的（大题）是**分析代码的题目。**」

> **他给的复习观与优先级**
>
> **① 先靠选择题拿分：**他布置的课下任务是复习排序这一章的选择题——**王道书上真题从第 13 题一直考到第 21 题**，「这边大概考了**七八个选择题**」，「隔个一两年就有一道」。
> **② 两个冷门大题点是堆排序与外部排序**；外部排序「这几年考的还是比较多的，之前的话是比较冷门的考点」。
> **③ 今年大纲只动了一处：「变动不大**，就是（数据结构）堆这一块**加了一个堆的应用**」——所以本讲堆讲得更细。
> **④ 必背的那张表：**「把书上的这一小节好好看一下：**各种内部排序算法的比较和应用**……时间复杂度、空间复杂度、稳定性，然后适用的存储（结构）。」「**这个表格肯定是要背的**」「基本要背个六七遍可能才能记牢，**一定要尽早开始。**」

本讲覆盖（对照你给的大纲）：插入（直接插入、折半插入、希尔）、交换（冒泡、快速）、选择（简单选择、堆排序）、归并、基数、复杂度与稳定性对比、快排划分与堆调整的大题推演——**你列的每一项他都讲了。**
你没列、他讲了不少的：① **堆的应用**（大纲新增，第七题）；② **外部排序整块**（磁盘块 / 缓冲区 / 置换选择排序 / 败者树 / 最佳归并树）；③ **学排序的六个角度**这套方法论；④ **哪些排序必须用数组**；⑤ 三路归并与有序段合并的双指针通法。

2 六个角度 + 三个前置概念

2.1 他给的六个角度（照这个清单逐个排序过一遍）

> 「你学习排序要从哪几个角度去学？第一个，要关注它的**趟数**；第二个，是**每一趟有什么特点**；然后第三个是这个**比较次数、移动次数**……第五个就是这个**时空复杂度**，然后第六个就是**稳定性。**」「这些角度其实都是真题出现过的。」「（末尾补充）我感觉还要再考虑一个**适用范围**，比如说适用的**存储结构。**」

> ⚠ **他的编号是 1、2、3、5、6——没有第 4 个**
>
> 讲稿从「第三个比较次数移动次数」直接跳到「第**五**个时空复杂度，**全讲没有再出现「第四个」**（逐字检索 0 次）。末尾另起一句补的**「适用范围 / 适用的存储结构」**极可能就是漏念的那一项，**但他没有明说**——本资料按六问——趟数 / 每趟特点 / 比较次数与移动次数 / 时空复杂度 / 稳定性 / 适用存储结构——呈现，并把最后一项标为「他后补的一句」。

2.2 三个前置概念

① 稳定性：「排完序之后有两个数字相同的，他们的相对位置有没有换」「有两个数值都是 5，但是实际上他们两个是不一样的，如果排完之后相对顺序换了就叫不稳定了」
→ 「稳定性其实主要通过书上记忆解决，就记那几个不稳定（的）」——因为「稳定性的考试你想去举反例，有时候不太好举得到」
② 内排序 vs 外排序：「（以）前学的都叫内部排序……就是说你一次性可以把所有的数据都读到内存，然后一块就排」；外部排序是「读不进去了，只能读一部分」
③ 分类：插入类（直接插入、希尔、折半插入）、交换类（冒泡、快排）、选择类（简单选择、堆排序）、分配类（基数排序）、归并单独一类

「你去练的时候**一定拿同一个序列去练，不要拿不同的序列去练**。」「课下把每一个排序的所有流程去手写一下……**这些细节其实印象就比较深了**。」**本讲全程用的就是同一个序列**：「（讲义上）举例子一般我也经常举这个 `6159847` 」——即 `[6, 1, 5, 9, 8, 4, 7]`（n = 7）。

3 六种内排序逐个推演（同一序列走到底）

按他的讲课顺序：直接插入 → 折半插入 → 希尔 → 冒泡 → 快速排序 → 简单选择。**堆排序、基数排序、二路归并**在 §4.5、§4.7、§4.8。

3.1 直接插入排序

「直接插入排序一定要注意：**它第一趟一定是从第二个开始的**。」「你到底从后往前比较还是从前往后比较，这个都无所谓……**一般情况下我们是默认按照从后往前比的**。」「第二趟的时候，5 比 6 小、比 1 大，**比较了两次**。」「8 是比 9 小，注意这个时候很多人可能会觉得比较一次就行了——**注意这里一定是两次**，因为 8 比 9 小、比 6 大，你肯定要比完之后比 6 大才会停，不然他就不知道应该是放在 6 后面。」「这个比较次数移动次数这种（形式）真题都是考过很多次的，一定要注意。」

趟	插入谁	比较	移动	本趟结束后的序列
第 1 趟	1（第 2 个）	1	1	[**1**, 6, 5, 9, 8, 4, 7]
第 2 趟	5（第 3 个）	2	1	[1, **5**, 6, 9, 8, 4, 7]
第 3 趟	9（第 4 个）	1	0	[1, 5, 6, **9**, 8, 4, 7]
第 4 趟	8（第 5 个）	2	1	[1, 5, 6, **8**, 9, 4, 7]
第 5 趟	4（第 6 个）	5	4	[1, **4**, 5, 6, 8, 9, 7]
第 6 趟	7（第 7 个）	3	2	[1, 4, 5, 6, **7**, 8, 9]

★ **独立验算**：六趟比较/移动 = (1,1)(2,1)(1,0)(2,1)(5,4)(3,2)，与老师口述**逐个一致**；合计比较 14、移动 9（合计为本资料统计）。

【六个角度】趟数：固定 n−1 趟（「7 个元素比了 6 趟」）
每趟特点：「第一趟是前两个有序，第二趟是前三个有序」（⚠ 注意不是「前两个」，见 §6.2 第一行）
最好 O(n)：每趟比较一次、不移动；平均/最坏 O(n²)：比较与移动都是 1+2+…+(n−1) = n(n−1)/2
空间 O(1)：「整个过程是在原来的数据本身上去操作的」　稳定：稳定（「后面那个 8 和前面相等就不往前放，就放在他后面就行了」）
存储：数组、链表都行 ——「如果是链表，你只需要直接改一下指针」「反而链表的移动次数要少一点，但是比较次数是一样的」

3.2 折半插入排序（他只讲了口径，没重推一遍）

「折半插入就不展开讲了，因为和刚才是一模一样的。」「折半插入需要**原始数据有序**，那刚好前面就已经有序了。」「所以（插）4 先和中间的 6 比，比 6 小，再和中间的 1 比，**这样两步就到位了**。」「所以**折半插入只会减少比较次数，但是不会减少移动次数**，因为你最后 4 放在 1 后面，照样是需要把 5、6、8、9 往后移动。」

★ 独立验算（第 5 趟插入 4，前面是 [1,5,6,8,9]）：折半查找 low=1,high=5 → mid=3 与 6 比（1 次）；
  4 < 6 → 查 [1,5]；mid=1 与 1 比（2 次）；4 > 1 → low>high 结束，插入位置 2
  ⇒ 恰好 2 次比较，与他说「两步就到位」一致（直接插入同一步要 5 次比较）；移动仍是 4 次 ✔
【六个角度】趟数、每趟特点、时空复杂度、稳定性均同直接插入；唯一差别是【比较次数更少】
（降到 O(n log n) 量级），但移动仍是 O(n²)，总时间不变

3.3 希尔排序：增量 + 分组，组内用直接插入

「希尔排序要注意有几个概念：一个是这个**增量**，然后一个是**分组**。」「**增量是几就说明有几组**。」「**所以希尔排序的组内用的是直接插入（排序）**。」「每趟的特点就是：**每间隔增量是有序的**。」「真题里面也考过让你反推增量：给了你一个排序的结果，让你去拆这个增量可能是几。」「**最后一趟的增量必须是 1**」「增量是一的话整个就是一组，**就退化到了直接插入排序**。」

序列 [6, 1, 5, 9, 8, 4, 7]（★ 每步均为独立推演，与他口述的分组结果一致）
d = 3（3 组）：组①位置 1,4,7 = {6,9,7} → 6,7,9（他念「六七九」）
　　组②位置 2,5 = {1,8} → 1,8（他念「一和八算第二组」）
　　组③位置 3,6 = {5,4} → 4,5（他念「五和四算第三组」）
　⇒ [6, 1, 4, 7, 8, 5, 9]　「这时候希尔排序的第一趟就结束了」
d = 2（2 组）：组①位置 1,3,5,7 = {6,4,8,9} → 4,6,8,9（他念「变成 4689」）
　　组②位置 2,4,6 = {1,7,5} → 1,5,7（他念「第二组就 157」）⇒ [4,1,6,5,8,7,9]
d = 1（整表直接插入）：[4,1,6,5,8,7,9] → [1, 4, 5, 6, 7, 8, 9]（他念「变成 145 6789」）
⚠ 第三趟虽按结果直接写，但「你在排的时候也得按照直接插入排序那个一趟一趟来」：先排 1、再排 6、再排 5。

稳定性的反例（他现场编的例子，本资料复盘）

他把原序列改了三处：第 2 个 1 换成 **9***（两个 9 不一样）、第 5 个 8 换成 **2**、第 7 个 7 换成 **10**，得 `[6, 9*, 5, 9, 2, 4, 10]`。

> d=3：组①{6,9,10} 已有序「6 9 10 是一组」；组②{9*,2} → 2,9*「9 杠和 2 一组就变成了 2、9 杠」；
> 组③{4,5} 已有序 ⇒ [6, 2, 4, 9, 9*, 5, 10]
> d=1：整表直接插入 ⇒ [2, 4, 5, 6, 9, 9*, 10]
> ★ 独立推演与他的结论一致：两个 9 的相对次序被换掉（9* 原在前、排完在后）——
> 「这时候他们两个相对顺序就换了……它是不稳定的」
> ⚠ 讲稿里他念的最终结果转写成「变成了 24 256 然后 9 9 杠 10」，数字崩坏；本资料按他给的反例独立推演得上面的结果，与他的结论自洽。

【六个角度】趟数：不固定（「取决于你增量是怎么选的，只要保证最后一趟是一」）
每趟特点：每间隔增量是有序的（反推增量题就看每隔几个是有序的）
比较/移动：按直接插入的口径算　　空间：O(1)「它就没有用辅助空间」
时间：⚠ 本讲未给 —— 「（希尔排序）时空复杂度上也没有分析，比较复杂就不管」〔教材补充，非老师口径：约 O(n^1.3)〕
稳定：不稳定（反例如上）　　存储：只能用数组 ——「你用链表就不行了」「它想要实现不相连远处的交换」

3.4 冒泡排序

「冒泡（时）你到底是小的从下往上冒，还是大的从上往下沉，这个是**没有严格规定的**，只不过一般来说我们都是把这个小的从下往上挪。」「七和四相比，小的要放在上面，这个不要交换……之后四和八比较需要交换……六和一相比要交换。」「所以它的比较次数**第一趟一定是 n-1**。」「移动次数就不固定了，因为很有可能一趟没有交换，也有可能交换了很多次。」「**如果某一趟没有交换就可以停了**……那就认为已经有序了。」

序列 [6, 1, 5, 9, 8, 4, 7]（竖着写，每趟从最下面两个开始往上比，小的往上走）
第 1 趟（比较 6 = n−1 次，交换 4 次）：(7,4)不换 (8,4)换 (9,4)换 (5,4)换 (1,4)不换 (6,1)换
⇒ [1, 6, 4, 5, 9, 8, 7]　「第一个元素就排好了」
第 2 趟（比较 5 = n−2 次，交换 3 次）：(8,7)换 (9,7)换 (7,5)不换 (5,4)不换 (6,4)换
⇒ [1, 4, 6, 5, 7, 9, 8]　★ 他原话「第二趟结束了就变成 1465798」——逐位吻合
第 3 趟（比较 4 次，交换 2 次）：(9,8)换 (8,7)不换 (7,5)不换 (6,5)换 ⇒ [1,4,5,6,7,8,9]
第 4 趟（比较 3 次，交换 0 次）：(9,8)不换 (8,7)不换 (7,6)不换 ⇒ 无交换，停
★「所以总共的话这个题目就是四趟」——与独立推演完全一致（比较 6+5+4+3 = 18 次，交换 9 次）

【六个角度】趟数：不固定（「最好情况下冒泡需要一趟（一开始就有序），最坏的话需要 n−1 趟」）
每趟特点：第 1 趟把最小的放最上面，第 2 趟把第二小的放到第二个位置，依此类推
比较次数：每一趟固定（n−1, n−2, n−3…），但总次数不固定（可能提前停）　　移动次数：不固定
最好 O(n)（一趟、比较 n−1 次）；平均/最坏 $O(n^2)$
★ 他专门对比过：「冒泡和直接插入他们都是 O(n)，但是算的方式不一样：一个是 1 趟 × (n−1)，一个是 (n−1) 趟 × 1」　空间 O(1)；稳定（「和上面相等你不交换就行了」）
存储：数组、链表都行（「因为它每次交换都在相邻的两个元素，还是比较好操作的」）

3.5 快速排序：双指针 + 枢轴，方向不能反

「先把第一个元素取出来，先取到外面了，**所以快排的话一定要从右往左开始**，因为你把第一个元素取出来了，所以你右边找一个比 6 小的可以放到这里，就不用担心覆盖。」「如果你拿最后一个元素作为枢轴，那就应该从左边开始。」「**快排有一个东西叫枢轴**……你每次和谁比都是和那个枢轴去比。」
（最该背的定义）「注意快排的第二趟是这样定义的：**你把左边再进行一下快排，右边再进行一下快排，两者都处理一下，叫第二趟**」「不是（说）左边叫第二趟、（处理）右边叫第三趟。」「快排的思想属于**分治法**。」「如果

> 快排的某一趟只有一个元素，就不需要去运算了」——「写快排代码，它那个递归的出口其实就是如果只有一个元素就停了。」

```
序列 [6, 1, 5, 9, 8, 4, 7]，枢轴 = 第一个元素（默认口径）
第 1 趟（整表划分，枢轴 6 先取出）：
    j 从右往左：7 > 6 过 → 4 < 6 ⇒ 把 4 填到左指针位置（左指针处已是空位）
    i 从左往右：1 < 6 过，5 < 6 过，9 > 6 ⇒ 把 9 填到右指针停留的位置
    j 再往左：8 > 6 过 → 两个指针相遇 ⇒ 把枢轴 6 填进相遇位置 ⇒ [4, 1, 5, 6, 8, 9, 7]
    ★ 独立验算：左 [4,1,5] 全 < 6、右 [8,9,7] 全 > 6 ✔「这时候左边都小，右边都大」
第 2 趟（左右两边各做一次快排才算一趟）：
    左 [4,1,5]（枢轴 4）：5 > 4 过 → 1 < 4 填到左指针位置 → 相遇填入 4 ⇒ [1,4,5]（他说「145」）
    右 [8,9,7]（枢轴 8）：7 < 8 填到左指针位置 → i 往右 9 > 8 填到右指针位置 → 相遇填入 8 ⇒ [7,8,9]
    ⇒ 全表 [1, 4, 5, 6, 7, 8, 9]
第 3 趟（递归出口）：「第三趟的时候他会发现 4 的左边只有一个元素，如果你只有一个元素一定是有序的，
    就没必要去快排了」—左右都只剩单元素，直接照搬。
```

```
【六个角度】趟数：不固定。最好「每次均匀分成两部分」⇒ log n 趟；最坏「你举一个一开始就有序的
  序列，第一个元素作为枢轴，那第一趟结束之后元素全在右边」⇒ n 趟
每趟特点：每趟产生一个枢轴，枢轴左边都小、右边都大
比较次数：只和枢轴比      移动次数：「交换的时候就发生了移动」（快排属交换类）
每趟开销：n —「两个指针相遇了……那说明他们一块走的路程就是一个 n」
  最好/平均 O(n log n)（= n × log n）；最坏 n+(n−1)+…+1 ⇒ O(n²)
  「严格意义上来说（最好时）应该是小于 n log n，但实际上你就当做每趟就是 n，然后 log n 趟」
空间：⚠ 本讲未给值 —「因为它是一个递归函数，它的空间复杂度取决于递归函数的深度」「等到后面
  讲到那个快排的代码到时候再看」〔教材补充，非老师口径：O(log n) ~ O(n)〕
稳定：不稳定（「这个稳定性就直接记一下吧，因为有时候去举反例不太好举」）
存储：只能用数组 —「会经常把这个不相连元素进行交换」「如果是单链表，他也没法从右往左走」
```

3.6 简单选择排序（王道书上没给图，但必须会手推）

> 「简单选择（排序）……**王道书上没有给那个图，其他排序王道书上是给这个示意图的，简单选择书上没给，但是也要会。**」「简单选择是他**选出一个最小的和第一个元素交换。**」「六和一比，一小，一再和五比，还是一小……**相当总共要比较六次。**」「所以虽然他也有交换，但是**他不属于交换类**，因为他最核心的步骤是选出最小的。」**（两句话最该记）**「这个简单选择**每一趟的比较次数是固定的，(总的)比较次数也是固定的**」「那一定是从 n−1 加 n−2 一直加到 1。」

趟	选出谁	比较次数	移动	本趟结束后的序列
第 1 趟	1（与第 1 个 6 交换）	n−1 = 6	1	[**1**, 6, 5, 9, 8, 4, 7]
第 2 趟	4（与第 2 个 6 交换）	n−2 = 5	1	[1, **4**, 5, 9, 8, 6, 7]
第 3 趟	5（本来就在第 3 位）	n−3 = 4	0	[1, 4, 5, 9, 8, 6, 7]
第 4 趟	6（与第 4 个 9 交换）	n−4 = 3	1	[1, 4, 5, **6**, 8, 9, 7]
第 5 趟	7（与第 5 个 8 交换）	n−5 = 2	1	[1, 4, 5, 6, **7**, 9, 8]
第 6 趟	8（与第 6 个 9 交换）	n−6 = 1	1	[1, 4, 5, 6, 7, **8**, 9]

★ **独立验算**：比较 6/5/4/3/2/1、移动 1/1/0/1/1/1 与口述逐个一致；合计比较 21 = n(n−1)/2、移动 5（合计为本资料统计）。

```
【六个角度】趟数：固定 n−1 趟      每趟特点：第 k 趟把第 k 小的放到第 k 个位置
比较次数：每趟固定、总数也固定 = n(n−1)/2 ⇒ O(n²)        移动：每趟 0 或 1 次 ⇒ 最多 n−1 次
时间：最好 = 最坏 = 平均 = O(n²)（「他无论最好最坏平均都是这么多次」）      空间 O(1)
稳定：不稳定（「这个还是看书上的那个写法吧，有时候举反例不太好举」）
```

4 堆的三个动作、堆排序、堆的应用、基数与归并

4.1 定义与考法分布

> 「任何一个节点如果比左右都大，这个叫**大根堆**；任何一个节点比（它的）每一个（孩子）都小，这个叫**小根堆**。」**他把历年堆的考法点了一遍（本讲最值钱的考情）**：「首先这个考了**堆的插入**……问你插入的时候这个**比较次数**；另外一个是问你**删除**的时候这个**比较次数——所以这个比较次数其实考的频率还挺高的**。然后另外的话就考你**建堆的过程**，再往后考一些**堆的概念**，再往后考**堆的插入**。」

4.2 堆的建立（从最后一个非叶子结点倒着调整）

> 「第一步，先把它写成**完全二叉树**」「第二步，倒着，然后**从第一个非叶子节点开始调整**」「7、4、8、9 是叶子节点……**所以从 5 号（结点）开始调整**。」「调整的时候注意，**注意是否需要左右比较**」「这个 5（结点）是需要左右比的……**这里比较了两次**。」
> **（本讲最重要的一条警告）**「这时候没完，要注意一个这个向下调整的过程——**要注意节点持续向下调整**」「因为这个 6 刚才被换下来，其实下面还有东西，还得看一下能不能再往下放」「**很容易做到这一步就漏了、就停了，一定要再往后走**」。

```
序列 [6, 1, 5, 9, 8, 4, 7] 建成大根堆（★ 逐步独立推演，与他念的中间结果逐位吻合）
先按层序写成完全二叉树（下标从 1 开始）：
              6(1)
          /          \
       1(2)          5(3)
      /    \        /    \
   9(4)   8(5)   4(6)   7(7)
非叶子结点 = 1、2、3（最后一个 = ⌊n/2⌋ = 3），倒着调：
① 调结点 3（值 5）：左右比 4 与 7 → 7 大（第 1 次）；7 与 5 比 → 换（第 2 次）
   ⇒ [6, 1, 7, 9, 8, 4, 5]      （他念「第二步是 617985」—讲稿漏了一个 4）
② 调结点 2（值 1）：左右比 9 与 8 → 9 大（第 3 次）；9 与 1 比 → 换（第 4 次）
   ⇒ [6, 9, 7, 1, 8, 4, 5]      （他念「第三步是 6971845」）
③ 调结点 1（值 6）：左右比 9 与 7 → 9 大（第 5 次）；9 与 6 比 → 换（第 6 次）
   ⇒ [9, 6, 7, 1, 8, 4, 5]      （他念「第四步是 9671845」）
④ 6 被换到结点 2 后【继续向下】比：左右比 1 与 8 → 8 大（第 7 次）；8 与 6 比 → 换（第 8 次）
   ⇒ [9, 8, 7, 1, 6, 4, 5]      （他念「最后一步是 9871645」，并说「最后就变成了 986」= 路径 9→8→6）
★ 比较次数合计 8 = 4 个结点 × 2（一次左右比 + 一次与待调元素比），与他口述完全一致
```

4.3 堆的插入：只往上比，不左右比、不向下调整

> 「新插入的结点你应该放到**最下面最右边**」「然后注意**插入只是往上比就行了，不需要左右比了**，因为左右比已经没有意义了。」「比方说 13 你往上比，13 和 6（比、）交换，这时候你 13 再和 8 比没有意义了——因为原来的话就是一颗堆，上面的 12 就是比 8 大……**所以 13 直接和 12 比。**」「**（12 被放下来）也不用往下比了，因为原来的话 12 就在上面（比下面的都大）。**」「**计算机内部肯定是比完才知道停的**，千万不要说到这一步你就停了、不处理了。」

例（他手画的图，讲稿只念出了路径）：插入 13
　13 插到最下面最右边 → ① 与父结点 6 比 → 13 > 6，交换（1 次）
　　② 与 12 比 → 13 > 12，交换、12 下沉（2 次）
　　③ 与 15 比 → 13 < 15，停（3 次）—「比完才知道不需要再往上走了」
　⇒ 共 3 次；旁支的 8 不参与比较
⚠ 讲稿中这棵堆的完整形态没念出来（只有路径 6 → 12 → 15 和旁边的 8），本资料只给出路径。
课后 16 题（大根堆插入 18）：① 18 与父结点 10 比 → 往上走　② 18 与 25 比 → 停 ⇒ 共 2 次
　干扰项：「有人可能就觉得到这里就停了，有人可能就会误选一个 1」「也有人可能会选一个 3（把 18 和 13 又比了一下）」—都是错的：「这个是堆的插入，不需要左右比，只往上比就行了」

4.4 堆的删除：最后一个结点顶上来，然后【持续向下】调整

> 「堆的删除（堆顶）结点或者任何一个结点都能删。」「（删堆顶就是）把最下面最右边的叶子结点**替换到删除位置**」「然后**从（这个）删除位置往下调整**，所以这时候就需要左右比了。」「换了位置之后这个要**持续往下调整**，并不是调一次就够了。」「如果你要删除的是非堆顶元素……（上面）这些位置你都没动，它本来就是一颗堆，就不需要调整的，**从删除位置往下走就行了。**」

例 1（删堆顶 15，用最后一个结点 2 顶上）：2 与（14、12 中较大的）14 比 → 14 与 2 换 → 继续向下：
　2 再与（10、12 中较大的）12 比 → 12 与 2 换 → 2 下面没结点了，停 ⇒ 又成一颗大根堆
例 2（删非堆顶元素 12，用最后一个结点 2 顶上）：只从删除位置往下：2 与（8、6 中较大的）8 比
　→ 8 与 2 换 → 继续向下：2 与（唯一剩下的）5 比 → 5 放上去 ⇒ 完成
课后 17 题（小根堆删 8，问比较次数）：
　① 根的两个孩子 15 与 10 比 → 10 更小（第 1 次）
　② 更小的 10 与顶上来的 12 比 → 交换（第 2 次）
　③ 12 只有左孩子 16、没有右孩子 → 只需 12 与 16 比 → 不用再调整（第 3 次）⇒「这应该是比三次」
　★ 独立复核：第 3 次不可省（要「比完才知道不需要再往下」）
⚠ 讲稿里这棵小根堆的完整层序转写成「815 2134 16」，数字崩坏；本资料只保留叙述能互印的部分（根 8、孩子 15 与 10、顶上来的 12、12 的左孩子 16），不替他把整棵树补出来。

4.5 堆排序：建堆 + 反复「删除堆顶」

> 「堆排序它只会用到**堆的建立和删除**，其他是用不到的。」「因为堆排序它属于**选择类排序**，它先把最大的选择出来，再把第二大的选择出来……所以你选出最大的之后就可以把它删除了。」「（每）一次排序从堆顶……这个高度是 log n……那总共就是 n 个元素，n log n。」「严格意义上来说是 n−1 个元素，因为你把 n−1 个排好，剩下一个就不用管了……但是都无所谓。」「它也不分最好、最坏、平均，都是这么多次。」

用上面建好的大根堆 [9, 8, 7, 1, 6, 4, 5]（★ 独立逐步推演）
① 删堆顶 9：最后一个结点 5 顶到根 ⇒ [5,8,7,1,6,4]；左右比 8 与 7 → 8 大；8 与 5 比 → 换
⇒ [8,5,7,1,6,4]；5【继续向下】：左右比 1 与 6 → 6 大；6 与 5 比 → 换 ⇒ [8,6,7,1,5,4]（第二大 8）
（他念「8 和 7 相比 8 更大，8 再和 5 相比……再往下还有节点，还得比 1 和 6 相比 6 大，
然后 6 再和 5 交换，这时候 5 下面没东西了就不用管了」）
② 删堆顶 8：4 顶到根 ⇒ [4,6,7,1,5]；左右比 6 与 7 → 7 大；7 与 4 比 → 换 ⇒ [7,6,4,1,5]（第三大 7）
③ 依此类推 ⇒ [1, 4, 5, 6, 7, 8, 9]
【六个角度】趟数：固定 n−1 趟；每趟特点：和简单选择一样（每趟选出一个当前最大的放到最终位置）
时间：最好 = 最坏 = 平均 = O(n log n)　　空间 O(1)　　稳定：不稳定
存储：只能用数组 —「不要觉得它是二叉树（就要用链表）」「你一个节点想要和父亲节点去比的时候，
（而）二叉树那个指针是单向的，父节点指向子节点，你想通过一个子节点定位到父节点，这个很麻烦」
★ 数组存堆的下标关系（他说的「二叉树的顺序存储」）：从 1 开始存（王道书默认）：父 = ⌊x/2⌋；
左孩子 = 2x；右孩子 = 2x+1。「3 号节点它的父节点就是 3 除 2 向下取整，那就是 1」
从 0 开始存也行—「你把根节点从 0 开始存还是从 1 开始存都无所谓，没有严格意义上的规定」

4.6 堆的应用（今年大纲新增，第七题：找最小的 10 个数）

「**你不要把整个（数组）排序，如果整个排序是 n log n，开销很大**……专门说让你实现思路就可以，并不让你写（代码）。」「**你要找最小的 10 个数，你是需要建的是大根堆，刚好反过来**！」「你先把前十个给它建到堆里面，之后从第十一个到第 n 个开始去遍历。」
「如果 x 比 A（堆顶）大——因为这（现在）是一颗大根堆，说明 x 比这十个都大，那说明 **x 就不是最小的十个数**，接下来处理下一个元素就可以了。」「若 x 小于 A，那说明 **A 就不是最小的十个数**……那你应该把 x 放到堆顶进行调整——**其实 x 放堆顶的意思就是把 A 删除了**，相当于执行了一次堆的删除操作，**只不过这次不是拿下面（最后结点）替换到上面，是拿当前的元素替换到堆顶。**」「**找最小的（10 个数）用大根堆，找最大的 10 个数就用小根堆。**」

【思路一：大根堆（他讲的思路）】
① 取前 10 个数建一个大小为 10 的大根堆（堆顶 = 这 10 个里最大的）
② 从第 11 个扫描到第 n 个，设当前元素 x、堆顶 A：x > A → 跳过（x 比堆里 10 个都大）；
x < A → 用 x 覆盖堆顶（等价于删掉 A），再向下调整成大根堆（等于号写哪边都行）
③ 结束后堆里剩下的 10 个就是最小的 10 个数
★ 时间：建堆 O(10) = 常数；后面 n−10 个元素最坏每个调到堆顶并向下调整 log 10 次
⇒ O(n + (n−10)·log 10) —「这个 log 10 是一个常数项，其实最后这个时间复杂度约等于 O(n)」
★ 空间 O(1)「它只消耗了常数项的空间，这个建堆的话只需要 10 个数组空间就可以了」
【思路二：10 个元素的数组 + 类似直接插入排序（答案给的第二种思路）】
① 用前 10 个数建一个 10 大小的数组，「按照插入排序的那个思想」把这 10 个排好序
② 从第 11 个开始，拿 x 与数组里最大的（有序数组最后一个 a9）比：x > a9 → 跳过；
x < a9 → 把 a9 删掉、x 放进去（直接覆盖），再做一次插入排序保持有序
★ 最坏每个元素要移动 10 次 ⇒ O(n × 10)；堆是 O(n × log 10) ⇒「堆排序（的开销）要更小一点」
但「你并不是 n−10 个元素都需要调整，所以他们两个开销并没有差别那么大，还是比较接近的」

他借这道题说的一句话：「查找和排序的本质基本就是离不开比较的」——「你去查找的时候肯定是不停地去比较，一直到成功或者失败。」

4.7 基数排序：分配 + 收集

「基数排序其实就先比较个位、再比较十位、再比较百位，这个概念比较简单，重点是它的**时间复杂度的分析**。」「它的特点就叫**分配和收集**，核心就记这几个字就可以了。」「（桶）这里是链表，所以你 252 去接收 213、223，并不是操作两次，直接操作一次就可以了……不管它的长度多长，只需要接一次。」「**你分配的时候每一趟都是**

> O(n)……**收集的开销是 O(r)**，这个 r 的意思就是你各位的取值是 0 到 9，一共十种可能。」「假如说现在即便有 100 个数，合并到一块也就是 10 条链表，合并开销是 10 次就可以了，并不需要 100。」「**总共还需要 d 趟**」。

```
例：排序 [213, 252, 223, 189, 315]（他先写 213/252/223/182/312，「因为全是 2 和 3 这个例子不太好」，
    把 182 与 312 换成了 189 与 315）—— ★ 逐趟独立推演，与他口述的收集顺序一致
第一趟（按个位分配 → 按 0~9 收集）：个位 3：213、223 | 2：252 | 5：315 | 9：189
   ⇒ [252, 213, 223, 315, 189]      「第一趟之后个位就有序了」
第二趟（按十位）：十位 1：213、315 | 2：223 | 5：252 | 8：189
   ⇒ [213, 315, 223, 252, 189]      「第二趟之后按照十位有序，十位相同就按个位有序」（也正是他念的串）
第三趟（按百位）：1：189 | 2：213、223、252 | 3：315 ⇒ [189, 213, 223, 252, 315] 全表有序
【时空复杂度】无论最好、最坏还是平均，都是 d × (n + r)；d = 趟数（= 关键字位数，三位数 3 趟）
   「n 是分配的开销，r 是收集的开销」      空间 O(r)（「你要构造一个大小为 r 的链表」）      稳定
```

4.8 二路归并排序（顺带三路归并）

> 「归并（排序）就没（每）两个一组，排个序，不够两个就自己（单独一组），这是第一趟。」「把两个有序序列合并成一个完整的（有序序列），就用了双指针。」「当你（其中一个）走完之后，把剩下的接过来，整个就有序了。」「所以你可以看出来归并排序这个趟数应该是 **log n 趟**，无论最好、最坏还是平均都是 log n 趟。」「每一趟的开销是 O(n)，因为每一趟都是要把这个所有的元素扫描一遍。」
>
> **（本讲唯一强调的「不一样」）**「注意**归并排序它是有空间复杂度的，有一个 O(n) 的空间复杂度**」「你不能直接写在原来数组本身……你如果把 4 直接替换到原来的位置，最后就找不到这个 5 了，这个 5 就被覆盖了。」「**其实整个这里面空间复杂度不是 O(1) 的，就只有快排（因为用到递归）、归并（有辅助空间）、基数排序（也是）有空间的，其他都是 O(1)**。」

```
序列 [6, 1, 5, 9, 8, 4, 7]（★ 独立推演，与他念的中间结果逐位吻合）
第一趟：[1,6] [5,9] [4,8] [7]            （他念「那变成了 16  59  487」）
第二趟：[1,5,6,9] [4,7,8]                （他念「所以就变成了 1569  478」）
第三趟：双指针合并：1 与 4 比 → 1；4 与 5 比 → 4；5 与 7 比 → 5；6 与 7 比 → 6；9 与 7 比 → 7；
   9 与 8 比 → 8；一段走完，把剩下的 9 接过来 ⇒ [1, 4, 5, 6, 7, 8, 9]
【六个角度】趟数：⌈log₂n⌉（n = 7 时是 3 趟）      每趟特点：每个有序段的长度翻倍
每趟开销 O(n)      时间都是 O(n log n)      空间 O(n)（辅助数组，本讲特别强调）
稳定：⚠ 本讲未说〔教材补充，非老师口径：归并排序稳定〕      存储：需要辅助数组，不能原地
三路归并（讲外部排序前的铺垫）：「三路归并就三个指针，从三个里面找出最小的」「谁小就把谁的指针往后走」
   「你可以把任意一个有序的（序列）合并成一个完整的有序序列」
   （⚠ 讲稿这一段的数值转写成「136/80/49/257」，不可辨读，本资料不复原题面）
```

5 外部排序（置换选择、败者树、最佳归并树）

> **为什么这一块要重点看：他给的考频判断**
>
> 「外部排序这几年考的还是比较多的，之前的话是比较冷门的考点。」「因为内部排序已经考了很多年，很多东西其实基本都考的差不多了，**所以最近外排序考的比较多，你要注意一下。」「外部排序用的是什么排序？用的是归并的思想**（这之前也出过一道（选择）题）。」「这个外排序这块确实是数据结构里面的一个难点。」

5.1 四个必须记住的名字 + 一次完整的归并（三个归并段、七个磁盘块）

> 「首先一个概念叫做**磁盘块**……外存最小的单位（就是）磁盘块。」「假设一个磁盘块能放三个数据。」「（实际中）一个磁盘块是 4KB，假如一个数据是 int 占 4B，那起码能放 1024 个……但为了方便理解我们都画的比较小。」「假如说现在你内存一次性只能读三个磁盘块，多了就放不下了」「这个叫**输入缓冲区**」「（输出端）叫**输出缓冲区**，输出缓冲区一般就一个。」
> 「通过三指针，这样一个流程和之前讲的一模一样」「**谁最小你先把它放到输出缓冲区**」「当这个输出缓冲区满的时候你把它输出」「（某个输入缓冲区的）磁盘块走完了……这时候你就可以把它清空，读一下（这个有序段的）第二个磁盘块」。「注意这个涉及频繁的读磁盘块、写磁盘块」。

```
用 §5.2 置换选择排序生成的三个归并段（每块放 3 个数据，共 7 个磁盘块）：
   段1：[37, 51, 63 | 92, 94, 99]                  2 块
   段2：[14, 15, 23 | 31, 48, 56 | 60, 90, 166]    3 块
   段3：[8, 17, 43 | 100]                          2 块
内存：3 个输入缓冲区（每段各读入 1 块）+ 1 个输出缓冲区（装 3 个数据就写回外存）
合并过程（★ 完整模拟，与他念的每一次「谁最小」「读哪个块」逐次吻合）：
   初始读入 段1{37,51,63}、段2{14,15,23}、段3{8,17,43}，三指针指在 37 / 14 / 8
   取 8（段3→17）、取 14（段2→15）、取 15（段2→23）          ← 输出缓冲满，写回外存
   取 17（段3→43）、取 23（段2块1用完，读块2 {31,48,56}）
   取 31、取 37（段1→51）、取 43（段3块1用完，读块2 {100}）
   取 48、取 51（段1→63）、取 56（段2块2用完，读块3 {60,90,166}）
   取 60、取 63（段1块1用完，读块2 {92,94,99}）、取 90
   取 92、取 94、取 99（段1用完）、取 100（段3用完）、取 166（段2用完）
   ⇒ [8,14,15,17,23,31,37,43,48,51,56,60,63,90,92,94,99,100,166]
   「现在内存明明只有三个磁盘块，但是它可以把（这）六七个磁盘块排好序，这个就（是）外排序的思想。」
★ 校验：段1 6 个 + 段2 9 个 + 段3 4 个 = 19 个数据，占 2+3+2 = 7 块 —— 与他说「总共占了七个
  磁盘块」「第一个归并段占两个磁盘块、第二个占三个、第三个占两个」完全一致
```

5.2 置换选择排序：怎么造出那三个有序的初始归并段

> 「如何得到这三个分别有序的归并段？因为你只有每个归并段有序了，才能去进行外部排序，才能用归并。这个概念叫做**置换选择排序**，这里就是记一些流程就可以了。」「先有几个概念叫做**工作区**，工作区能保留 M 条记录。」**规则（最该记的流程）**：「先往工作区里面放（M 个）数字，之后把这里面的最小的取出来」；「有一个变量记录一下**当前归并段的最大值**」「（后来）最小的元素是 14，14 没有 37 大，你就不能用 14，再来找第二小的……51 是比 37 大，就可以放在外面，**因为归并段每个归并段得是有序的**」；「**每往外拿出一个，再补一个新的（读进来）**」「工作区始终是 M 个」；「如果（工作区里）4 个都没有（当前最大值）大，那说明第一个归并段就可以结束了」。

```
工作区 M = 4；输入序列（按他口述的读入顺序复原）：
   51, 94, 37, 92, | 14, 63, 15, 99, 48, 56, 23, 60, 31, 17, 43, 8, 90, 166, 100
先读 4 个进工作区 {51,94,37,92}，取最小 37 输出；读入 14 补位；当前段最大值 = 37
   {51,94,92,14}：14 < 37 不能用 → 取次小的 51（≥37）输出，最大值 51；读入 63
   {94,92,14,63}：14 < 51 → 取 63 输出，最大值 63；读入 15
   {94,92,14,15}：14、15 都 < 63 → 取 92 输出，最大值 92；读入 99
   {94,14,15,99}：取 94 输出，最大值 94；读入 48
   {14,15,99,48}：只有 99 ≥ 94 → 取 99 输出，最大值 99；读入 56
   {14,15,48,56}：4 个都 < 99 ⇒ 第一归并段结束
★ 归并段 1 = 37, 51, 63, 92, 94, 99      （他念「3751 623 然后 929499」）
第二段（工作区剩 {14,15,48,56}，最大值重新开始）：
   取 14（读入 23）→ 取 15（读入 60）→ 取 23（读入 31）→ 取 31（读入 17）
   → 17 < 31 不能用，取 48（读入 43）→ 17、43 < 48，取 56（读入 8）
   → 取 60（读入 90）→ 取 90（读入 166）→ 取 166（读入 100）→ 剩 8、17、43、100 都 < 166 ⇒ 结束
★ 归并段 2 = 14, 15, 23, 31, 48, 56, 60, 90, 166（他念「141523」「314856」「6196166」）
★ 归并段 3 = 剩下的 {8, 17, 43, 100}（他念「8、7、43、100」——7 应为 17，见 §附）
【另一个常考结论】「如果你的元素一开始就是完全有序的，那你放到工作区，这个第一个归并段总能
   往后增加，这第一个归并段最大值就是 n」——★ 复核：完全有序时只会生成【1 个】归并段，长度 = n。
```

5.3 败者树：把 k 路归并「每次找最小」从 k−1 次压到 log k 次

「**败者树是用来解决**（从 k 个归并段里）**找出最小值**（的）」「如果你是八路归并……要比较七次」「就像 k 个归并段，你得比较 k−1 次……接下来找出第二个（最小）的，还得比较 k−1 次」「这个开销就很大，**所以败者树是用来解决这个问题的。**」「**谁失败的话就把谁放在父亲节点**」「**这个节点里面存的是编号，是（这个）归并段的编号，不是这个 27、12 本身**」「（这）就有点像那个（打）擂台赛一样，谁赢了再往上一层和另一个人去比。」「这个节点叫做**冠军节点**。」

「**（调整时）只和上面比就行了**」「你 3 号（段）都（已经）比 2 号小了，肯定也比 1 号小，就没必要和 1 号比了——**这个败者树，每次是只和父亲节点去比就行了。**」「所以这样调整的话，每次只需要调整这个**（树）高度次**」。「通过这种方式从 n 个元素里面找最小的，每次的开销是一个 log n（log k），要比之前的（k−1）……这样就更快了。」

```
八个归并段的段首：① 27   ② 12   ③ 1   ④ 17   ⑤ 2   ⑥ 9   ⑦ 11   ⑧ 4
（★ 按他口述的比赛过程独立重建，与他念的每一个「谁失败 / 谁赢了往上走」逐条吻合）
第一次建树：(①27,②12)→27 失败，存【①】 (③1,④17)→17 失败，存【④】，③ 胜出
            (⑤2,⑥9)→9 失败，存【⑥】，⑤ 胜出      (⑦11,⑧4)→11 失败，存【⑦】，⑧ 胜出
  半区：(②12,③1)→12 失败，存【②】，③ 胜出        半区：(⑤2,⑧4)→4 失败，存【⑧】，⑤ 胜出
  决赛：(③1,⑤2)→2 失败，存【⑤】⇒ 冠军 =【③】「最终你能得出 3 号段是最小的」
第二次（3 号段取出 1 后补入 6）——只沿路径往上比：
   3(6) 与 ④(17) 比 → 3 胜；3(6) 与 ②(12) 比 → 3 胜；3(6) 与 ⑤(2) 比 → 5 胜
   ⇒ 冠军 = 5 号段，第二小 = 2     「为什么只和 2 号比、不用和 1 号比？」——节点存的【②】就是上一轮
   1 号与 2 号比出来的败者，「你都比 2 号小了，肯定也比 1 号小」
第三次（5 号段取出 2 后补入 3）：5(3) 与 ⑥(9) 比 → 5 胜；5(3) 与 ⑧(4) 比 → 5 胜；
   5(3) 与 ③(6) 比 → 5 胜 ⇒ 冠军 = 5 号段，第三小 = 3
★ 对比：不用败者树每次要 k−1 = 7 次比较（他逐次演示：27与12比、12与6比、6与17比、6与2比、
   2与9比、2与11比、2与4比 ⇒ 正好 7 次）；用败者树每次只需树高 = log₂8 = 3 次
「冠军（节点）存的是这个归并段的编号，整个这个败（者树）存的都是编号，最下面的叶子（节点）存
 的（才）是数值」
```

5.4 最佳归并树：本质是哈夫曼树

> 「（如果）你现在采用二路归并……（先合并两个段）相当于这总共占五个磁盘块，你把这五个磁盘块都读进来一次、再写回去一次」；「（再）拿着五个有序的磁盘块和剩下的这两个磁盘块再去归并……把这七个都读进来一次、再写出去一次」；「**这个过程其实就是最佳归并树，就是之前学的哈夫曼树**」「你如果想要总的（读写）磁盘块最少，你肯定先把（长度）为二的磁盘块合到一块，再把这个长度为三的和这个合到一块」「**如果你（是）k 路归并，那你就每次应该选 k 个最小的**」「这个最佳归并树你要知道它本质上就是一棵哈夫曼树就可以了。」

```
三个归并段的长度（按磁盘块数）：段1 = 2、段2 = 3、段3 = 2，共 7 块（★ 次数为独立计算）
方案 A（先合并 段1(2) 与 段2(3)）：合并成 5 块：读 5 + 写 5 = 10；再与段3(2) 合并成 7 块：
    读 7 + 写 7 = 14 ⇒ 合计 24 次磁盘块读写
方案 B（哈夫曼：先合并两个最短的 2 + 2 = 4）：读 4 + 写 4 = 8；再与 3 块段合并成 7 块：
    读 7 + 写 7 = 14 ⇒ 合计 22 次，比方案 A 少 2 次
    ✅ 与他的原话「这里是读写 8 个磁盘块，这里是读写 14 个，要比原来的话要小一点」一致
```

6 解题方法论与易错点清单

6.1 拿到任何一个排序，按这六问写就能拿分

```
① 趟数：固定吗？ 固定 n−1：直接插入、简单选择、堆排序；固定 d 趟：基数排序
    不固定：冒泡（可能提前停）、希尔（看增量）、快排（看划分是否均匀）
② 每趟特点：直接插入 = 前 k+1 个有序 | 冒泡、简单选择、堆排 = 第 k 小的就位
    快排 = 每趟产生一个枢轴、左小右大 | 希尔 = 每间隔增量有序 | 归并 = 段长翻倍
③ 比较次数：是【固定】还是【不固定】—简单选择每一趟固定、总的也固定；
    冒泡每一趟固定，但总的（因为可能提前停）不固定；快排只和枢轴比
④ 移动次数：简单选择每趟 0 或 1 次；快排交换即移动；冒泡不固定
⑤ 时空复杂度：最好 / 平均 / 最坏（只有归并、堆排、基数、简单选择是三者同阶）
⑥ 稳定性 + 适用存储结构：必答「数组 or 链表」这一问（下面这张对照就是考点）
```

排序	稳定？	适用的存储结构（他的原话）
直接插入	稳定	**数组、链表都行**：「如果是链表，你只需要直接改一下指针就行」
折半插入	稳定（他说「其他的都一样」）	**只能顺序存储**——「折半插入也得是顺序存储」（折半查找要随机访问）
希尔	不稳定（有反例）	**只能数组**：「希尔排序你用链表就不行了」
冒泡	稳定	**数组、链表都行**：「因为它每次交换都在相邻的两个元素」
快速排序	不稳定	**只能数组**：「不相连元素交换很麻烦」「单链表没法从右往左走」
简单选择	不稳定	通常数组（他没单独说）
堆排序	不稳定	**只能数组**：「要定位父节点 ⌊x/2⌋，链表是单向的，弄不了」
基数排序	稳定	链表（分配 / 收集用链表，空间 O(r)）

排序	稳定？	适用的存储结构（他的原话）
二路归并	稳定〔教材补充，非老师口径〕	需要 O(n) 辅助数组

> **他点名的「必须用数组实现」的三个（真题考过）**
>
> 「有这样的一道真题，它是说必须用数组去实现的排序，那就（是）希尔排序、快排，还有后面要讲的一个堆排序，**这三个必须用数组去实现，用链表的话效率都会很低**。像其他的像冒泡、直接插入，你用链表实现其实和数组没啥区别。」

6.2 易错点清单（他警告过的都在这）

容易搞错的地方	为什么错	正确做法 / 他的原话
以为「第二趟之后前两个有序」	把直接插入的趟号数错	「**第一趟是前两个有序，这个不要搞混了，第二趟是前三个有序**」——真题套路：给「第二趟排序后的结果」问可能是什么排序，「如果有一个选项说直接插入排序一定是错的」
直接插入里「比一次就停」	忘了比到「不大于」才停	「8 比 9 小……**注意这里一定是两次**，因为 8 比 9 小、比 6 大，你肯定要比完之后比 6 大才会停」
以为折半插入能减少移动	把比较和移动混一起	「折半插入**只会减少比较次数，但是不会减少移动次数**」（⚠ 他后一句「区别就是移动次数不一样」与本句矛盾，见 §附）
希尔最后一趟增量不取 1	整表不保证有序	「最后一趟的增量必须是 1」——「增量是 1 的话整个就是一组，就退化到了直接插入排序」
把简单选择当成交换类	它确实有交换	「虽然他也有交换，但是**他不属于交换类**，因为他最核心的步骤是选出最小的」
快排第二趟只算一边	趟的定义	「**把左边再进行一下快排，右边再进行一下快排，两者都处理一下，叫第二趟**」
快排枢轴取第一个却从左往右先走	会覆盖还没用到的数据	「先把第一个元素取出来……**所以一定要从右往左开始**」；拿最后一个元素做枢轴才「从左边开始」
建堆时「调到根就停」	漏了「持续向下调整」	「**要注意节点持续向下调整**」「很容易做到这一步就漏了、就停了，**一定要再往后走**」（他标为最容易丢分的一处）
堆插入时去左右比	把插入和删除搞混	「插入只是**往上比**就行了，**不需要左右比**」；「也不要到时候写个 13 和 8 去比，这种都是没有意义的」
堆插入时向下调整	原始已是一颗堆	「12 被放下来也不用往下比，因为原来的话 12 就在上面（比下面的都大）」
堆删除只调一次	与建立同理	「从删除位置往下调整……**是需要持续向下调整的**，并不是调一次就够了」

容易搞错的地方	为什么错	正确做法 / 他的原话
找最小 10 个数建小根堆	方向搞反	**「你要找最小的 10 个数，你是需要建的是大根堆，刚好反过来」**；找最大用小根堆
归并排序空间记成 O(1)	唯一有 O(n) 辅助空间的	**「归并排序它是有空间复杂度的，有一个 O(n) 的空间复杂度」**「直接写在原来数组本身，5 就被覆盖了」
置换选择排序漏「补新元素」	工作区要始终保持 M 个	「每拿外面一个，再新读进来一个」**「工作区始终是 M 个」**；比当前段最大值小的元素要留着不动
败者树节点存数值	存的是段的编号	「这个节点里面存的是编号，是归并段的编号」；「最下面的叶子节点（才）存的是数值」

7 考场速查卡

【〇】本讲的「万能样例」：序列 [6, 1, 5, 9, 8, 4, 7]（n = 7）在六种排序里的结果
直接插入：1,6 → 1,5,6 → 1,5,6,9 → 1,5,6,8,9 → 1,4,5,6,8,9 → 1,4,5,6,7,8,9
冒泡　　：[1,6,4,5,9,8,7] → [1,4,6,5,7,9,8] → [1,4,5,6,7,8,9] → 第 4 趟无交换停（共 4 趟）
快速排序：[4,1,5,6,8,9,7] → [1,4,5,6,7,8,9] → 第三趟只剩单元素（递归出口）
简单选择：[1,6,5,9,8,4,7] → [1,4,5,9,8,6,7] → … → [1,4,5,6,7,8,9]（每趟比较 6,5,4,3,2,1）
希尔　　：d=3 → [6,1,4,7,8,5,9]；d=2 → [4,1,6,5,8,7,9]；d=1 → [1,4,5,6,7,8,9]
归并　　：[1,6][5,9][4,8][7] → [1,5,6,9][4,7,8] → [1,4,5,6,7,8,9]（3 趟 = ⌈$\log_2 7$⌉）

【一】八种内部排序总表（★ 他本讲明确说出口的都在正文引了原话；标「补」的是教材口径）

排序	最好	平均	最坏	空间	稳定	存储
直接插入	O(n)	O(n^2)	O(n^2)	O(1)	稳定	数组/链表
折半插入	O(n)	O(n^2)	O(n^2)	O(1)	稳定	仅顺序
希尔	补 O(n^1.3)	补	补	O(1)	不稳定	仅数组
冒泡	O(n)	O(n^2)	O(n^2)	O(1)	稳定	数组/链表
快速排序	O(nlogn)	O(nlogn)	O(n^2)	O(logn)补	不稳定	仅数组
简单选择	O(n^2)	O(n^2)	O(n^2)	O(1)	不稳定	通常数组
堆排序	O(nlogn)	O(nlogn)	O(nlogn)	O(1)	不稳定	仅数组
二路归并	O(nlogn)	O(nlogn)	O(nlogn)	O(n)	稳定补	需辅助数组
基数排序	O(d(n+r))	同左	同左	O(r)	稳定	链表

⚠ 他的空间复杂度口径：「空间复杂度不是 O(1) 的就只有快排（用递归）、归并（有辅助空间）、基数排序（有空间），其他都是 O(1)」

【二】趟数：三个【固定 n−1】+ 一个【固定 d】+ 三个【不固定】
固定 n−1：直接插入、简单选择、堆排序　　固定 d 趟：基数排序（d = 关键字位数）
不固定：冒泡（有序时 1 趟就停）、希尔（看增量怎么选）、快速排序（均匀 log n 趟、最坏 n 趟）

【三】堆的三个动作（今年大纲新增堆的应用，这块最可能出题）
建堆：① 先写成完全二叉树 ② 从最后一个非叶子节点 ⌊n/2⌋ 倒着往下调 ③ 每次先【左右比】选出较大的孩子，再与待调元素比 ④ 【必须持续向下】
　　例：[6,1,5,9,8,4,7] → [9,8,7,1,6,4,5]，比较 8 次（4 个结点 × 2）
插入：放到【最下面最右边】→ 【只往上比】，不左右比、不向下调整
　　例：插入 13 依次与 6、12、15 比 ⇒ 3 次；课后 16 题插入 18 依次与 10、25 比 ⇒ 2 次
删除：用【最后一个结点】替换到删除位置 → 从删除位置【持续向下】调，需要【左右比】
　　例：课后 17 题小根堆删 8 ⇒ 15与10比、10与12比、12与16比 ⇒ 3 次
堆排序：建堆 + 反复删除堆顶，n−1 趟，每趟 log n ⇒ 都是 O(n log n)，空间 O(1)，不稳定，仅数组
堆的应用（第七题）：找最小的 10 个数 ⇒ 建【大】根堆（堆顶是这 10 个里最大的）；x > 堆顶 → 跳过；
　　x < 堆顶 → 覆盖堆顶并向下调整；时间 O(n)、空间 O(1)；找最大的 k 个数则相反（用小根堆）

【四】外部排序一页通
磁盘块（外存最小单位）→ 输入缓冲区（k 个）/ 输出缓冲区（一般 1 个）
外部排序 = 归并思想；两个阶段：① 生成初始归并段（置换选择排序）② 多路归并（败者树加速）
置换选择排序：工作区保留 M 条记录；每次取【不小于当前归并段最大值】的最小元素输出，并从输入补一个新元素进工作区；工作区里所有元素都比当前最大值小 ⇒ 本段结束；元素完全有序时只生成 1 个段
败者树：解决「k 路归并每次找最小要 k−1 次比较」这个问题；内部节点存【败者编号】，叶子存数值，根上面是【冠军节点】；每次调整只沿路径比，开销 = 树高 = $\log_2 k$
最佳归并树：本质是【哈夫曼树】——每次先合并 k 个最短的段，读写磁盘块总数最少
　　例：段长 2、3、2 ⇒ 先合 (2+3)=5 再合 5+2=7：读写 10+14 = 24 次
　　先合 (2+2)=4 再合 4+3=7：读写 8+14 = 22 次（更优）

如果只记六件事

① **直接插入第一趟排第 2 个，第二趟之后是「前三个有序」**——最容易设陷阱的一句；
② **折半插入只减比较次数、不减移动次数**；希尔组内是直接插入，最后一趟增量必须是 1，不稳定；
③ **快排「第二趟」= 左右两边都各做一次划分**；第一个元素做枢轴必须从右往左先走；
④ **堆三件事的比较次数口径**：建堆 / 删除要左右比 + 持续向下，插入只往上比；堆排序只能数组；
⑤ **找最小的 k 个数用大根堆**（O(n)），找最大的用小根堆；
⑥ **空间不是 O(1) 的只有三个**：快排（递归栈）、归并（O(n) 辅助）、基数（O(r)）。

附：本资料的来源与边界

来源	内容	可靠性与边界
B 站《408 各科大题知识点强化课》第 4 讲「数据结构 · 排序专题」，分 P6，音频 **3 小时 28 分 44 秒**	§1 – §7 的全部	本地 Whisper 转写，讲稿 `.md` 3148 行 / 约 14.3 万字节，覆盖 `[00:00:29]`–`[03:28:00]`，即**全程**；其中 `[01:47:11]`–`[01:57:52]` 是课间休息（约 10 分 40 秒），另有若干安静读题时段（`[01:19:30]`、`[01:29:54]`、`[01:59:43]`、`[02:34:37]` 等，合计约 8 分钟无口述）。**全程无板书**：所有序列、堆、归并段都是口述 + 现场手画（画面看不到），本资料按他念出的数值逐步复原并**每一步都独立重算过**（见下两张表）。
教材 / 通用常识（本资料的补充）	标「补」的三处：希尔的时间复杂度、快排的空间复杂度、归并的稳定性	**教材补充，非老师口径。**他分别说了「比较复杂就不管」「（这个）等到后面讲那个快排的代码到时候再看」「（没说）」，本资料只给出教材结论并标注，**没有展开推导**。

★ **第五步自查（一）：算式独立验算 —— 他念出的每一个过程都重跑了一遍**

过程（出处）	讲稿里的说法	独立验算结果
直接插入 6 趟比较/移动（§3.1）	(1,1)(2,1)(1,0)(2,1)(5,4)(3,2)	✓ **逐趟吻合**（合计比较 14、移动 9）；最好 n−1 次比较、0 次移动 ✓
折半插入插 4「两步到位」（§3.2）	先与中间 6 比、再与中间 1 比	✓ [1,5,6,8,9] 中折半查找 4：mid=3→6，mid=1→1，结束 ⇒ 恰好 2 次 ✓
希尔 d=3 / d=2 / d=1（§3.3）	六七九、一八、四五；4689、157；145 6789	✓ [6,1,4,7,8,5,9] → [4,1,6,5,8,7,9] → [1,4,5,6,7,8,9]，逐组吻合
希尔稳定性反例（§3.3）	两个 9 相对次序换了（最终结果转写崩坏）	✓ [6,9*,5,9,2,4,10] → d=3 → [6,2,4,9,9*,5,10] → d=1 → **[2,4,5,6,9,9*,10]**，9* 由前变后 ⇒ 不稳定 ✓
冒泡四趟（§3.4）	[1,6,4,5,9,8,7]；**「1465798」**；第四趟无交换	✓ **逐位吻合**，第二趟结果 `1465798` 完全一致；比较 6+5+4+3=18，交换 9 次；共 4 趟 ✓
快排第一/第二趟（§3.5）	[4,1,5,6,8,9,7]；左 145、右 789	✓ 划分后左 [4,1,5] 全 < 6、右 [8,9,7] 全 > 6；第二趟两边各划分一次 ⇒ [1,4,5,6,7,8,9] ✓

过程（出处）	讲稿里的说法	独立验算结果
简单选择六趟（§3.6）	比较 6,5,4,3,2,1；移动 1,1,0,1,1,1	✓ **逐个吻合**；合计比较 21 = n(n−1)/2 ✓、移动 5；第三趟最小值 5 原位 ⇒ 0 移动 ✓
建大根堆四步（§4.2）	617985 → 6971845 → 9671845 → 9871645，比较 8 次	✓ → [6,1,7,9,8,4,5] → [6,9,7,1,8,4,5] → [9,6,7,1,8,4,5] → **[9,8,7,1,6,4,5]**，4 个结点 × 2 = 8 次 ✓（讲稿「617985」漏一位，应为 6179845）
堆排序的删除调整（§4.5）	删 9 得第二大 8；删 8 得第三大 7	✓ 删 9：5 顶上 → 与 8 换、再与 6 换 ⇒ [8,6,7,1,5,4]；删 8：4 顶上 → 与 7 换 ⇒ [7,6,4,1,5] ✓
堆插入 13（§4.3）	依次与 6、12、15 比	✓ **3 次**：13>6 换、13>12 换、13<15 停；旁支 8 不参与 ✓（树形未念出，只保留路径）
课后 17 题小根堆删 8（§4.4）	「这应该是比三次」	✓ **3 次**：15 与 10 比、10 与 12 比、12 与 16 比；第 3 次不可省 ✓
基数排序三趟（§4.7）	[252,213,223,315,189] → 213,315,223,252,189 → 有序	✓ **逐趟吻合**；最终 [189,213,223,252,315] ✓；d(n+r) = 3×(5+10) ✓
二路归并三趟（§4.8）	16 59 487；1569 478；双指针合并	✓ [1,6][5,9][4,8][7] → [1,5,6,9][4,7,8] → [1,4,5,6,7,8,9] ✓；趟数 ⌈$\log_2 7$⌉ = 3 ✓
置换选择排序的 19 个数据（§5.2）	段1 = 37,51,63,92,94,99；段2 = 14,15,23,31,48,56,60,90,166；段3 = 8,17,43,100	✓ **三段完全一致**（按工作区 M=4 + 记录当前段最大值的规则，模拟输入 51,94,37,92,14,63,15,99,48,56,23,60,31,17,43,8,90,166,100）；19 个数据 = 三段之和 ✓
三个归并段占几个磁盘块（§5.1）	「总共占了七个磁盘块」，分别占 2、3、2 块	✓ ⌈6/3⌉=2、⌈9/3⌉=3、⌈4/3⌉=2，合计 7 ✓；与归并过程的输出顺序逐次吻合
败者树三次冠军（§5.3）	冠军 3 号段(1) → 5 号段(2) → 5 号段(3)	✓ **三次全部吻合**：建树时 (1,2)(3,4)(5,6)(7,8) 败者为 1、4、6、7，半区败者 2、8，决赛败者 5，冠军 3 号段 ✓；树高 $\log_2 8$ = 3 ✓
最佳归并树读写次数（§5.4）	「这里是读写 8 个，这里是读写 14 个，要比原来的话要小一点」	✓ 方案 A（先 2+3）：读 5 写 5 + 读 7 写 7 = **24**；方案 B（先 2+2）：读 4 写 4 + 读 7 写 7 = **22** ⇒ 22 < 24 ✓
堆的应用第七题（§4.6）	建 10 个的大根堆；O(n)、O(1)	✓ 建堆 O(10) 常数 + (n−10) × 最坏 $\log_2 10$ ≈ 3.32 ⇒ O(n) ✓；第二种思路 (n−10)×10 ⇒「堆的开销要更小一点」✓

结论：讲稿里的过程性推算**没有发现算错**（这一讲与前面几讲不同，他的逐趟推演全部正确）；问题集中在**转写**与**一处自相矛盾的口误**上。

★ 第五步自查（二）：引语回对 —— 转写含糊、按上下文推断或存疑处

① [00:20:27]「折半插入和直接插入，**他们的区别就是移动次数不一样**」——与他前一句「**只会减少比较次数，但是不会减少移动次数**」**直接矛盾**。按教学口径与教材，两者差别在**比较次数**：本资料按前一句呈现，并声明后一句为**老师口误**（或转写把「比较」听成「移动」）。

② [00:13:33]「1 加 2 加 3 一直到第 n 减 1 趟，那就比较 **n 减一次**移动 **n 减一次**」——数字崩坏；按同段「还要乘一个 2，这样算出来就是 O(n²)」反推应为 **n(n−1)/2 次**，本资料按此呈现（§3.1）。

③ [01:12:49] 建堆中间结果「第二步是 **617985**」只有 6 位，按该步「7 和 5 换位置」应为 6179845（漏一个 4）。已按正确值呈现。

④ [01:28:01] 课后 17 题小根堆层序转写成「**815　2134　16**」，数字崩坏且与「12 只有左孩子 16」矛盾；本资料只保留叙述能互印的部分（根 8、孩子 15/10、顶上来的 12、12 的左孩子 16），**不替他补整棵树**。

⑤ [01:31:57]「**18 年**出了一道题目是 **6159817**，到了 **21 年**又出了题目，还是 **616915587**」——两串数字都崩坏（4 常被转成 1）；结合 [01:07:23]「举例子一般我也经常举这个 **6159847**」，18 年那串应为 6159847；21 年那串无法可靠复原，只保留两题很像、但一个考建立一个考插入这个结论。

⑥ [02:57:53]「第三个归并段，就是 **8、7、43、100**」——「7」应为 **17**（与「第二小的是 17」互印）；已按 17 呈现。[02:56:14] 第一归并段「3751 623 然后 929499」按模拟为 **37、51、63、92、94、99**；[02:59:27] 第二段「141523 / 314856 / 6196166」→ **14,15,23 / 31,48,56 / 60,90,166**。

⑦ [03:06:22]–[03:07:04] 败者树例子里八个归并段的完整内容整段崩坏——本资料**只保留比赛过程能唯一确定的八个段首值**（27、12、1、17、2、9、11、4），不猜各段后续数据。

⑧ [02:29:06] 三路归并的例子转写成「136 / 80 / 49 / 257」，不可辨读；只按他口述的比较顺序（取 1、2、3、4、5）复原了一个示意例，并标注**不作为题面**（§4.8）。

⑨ [00:12:02]「这里是三个书」、[00:12:19]「那个有学是前三个有学」——应为「（第二趟是）前三个**有序**」；[00:05:12]「第**五**个就是时空复杂度」——第四个角度缺失（§2.1 已说明）。

⑩ 他把「枢轴」念成「输走 / 数轴 / 书轴」、把「磁盘块」念成「吃饭块 / 尺寒框」——都有上下文算式跟着，已统一校正。

真题年份认定与依据（可标年份，但要说明凭什么）

· 2018 「这个 **18 年**出了一道题目（是）6159847」——**依据：老师口述年份 + 序列**，本资料未核实原卷（他念的串崩坏为 6159817，按同一序列复原）。

· 2021 「到了 **21 年**又出了题目……两道题很像，但实际上两道题不一样，一个考建立，另外一个是考插入」——**依据：老师口述**；该年份题面数字不可辨读，**不补**。

· 2024 「**去年**的（真题）考了一个选择题叫败者树」「就是去年的 **24 年**的真题」——两处独立说到，**成立度最高**；但也只说「考了败者树的概念」，**没有题面**（§5.3 的败者树例子是他自己画的，不是原题）。

· 年份不详 「（哪些排序）必须用数组去实现」（他说「也是考过的」）、「真题里面也考过让你反推增量」（希尔）、「（外部排序用归并）这之前也出过一道（选择）题」、「一道排序的（大题）是分析代码的题目」——**均未说年份**。

认定原则：凡标年份的一律注明是**老师口述、没有回原卷核对**；凡他只说考过 / 比较新 / 有一道的，一律标「年份不详」，不靠「看着像」定年。

老师没有讲 / 本讲没有覆盖的内容（对照你给的大纲与课程全貌）

① 你大纲里的六项，本讲全部讲了：直接插入、折半插入、希尔、冒泡、快速排序、简单选择、堆排序、归并、基数、复杂度与稳定性对比、排序大题（快排划分 / 堆调整）——**没有一项是「本讲未讲」**。
② 他明确「不讲」或留到后面的：·**快排与归并的代码**——「像快排和归并（排序）的代码其实是需要（你）掌握的，后期到时候再单独讲一下」（本讲只讲手推流程，**没有写一行代码**）；·**希尔的时间复杂度**——「比较复杂就不管」；·**快排空间复杂度的具体值**——「等到后面讲快排的代码到时候再看」。
③ 他点名的「数据结构还没讲」清单（讲末原话）：「像这个**散列表**，因为时间有限这个都没有展开讲」；「**KMP 以及改进**是另外一个」；「包括**折半查找**还有一些东西，这些其实都没有讲」；「包括 **B 树和 B+ 树**，它是一种选择题的高频考点……这些都没有讲」「下次上课的话就直接开始组成原理了」。
④ 全文没有出现的名字（逐字检索 0 次）：「锦标赛排序 / 树形选择排序」「桶排序」「计数排序」「多路平衡归并」「外部排序的 I/O 时间公式推导」——**本资料不替他补**。

本资料与你的大纲的差异（一句话版）

你列了、老师没讲的：无——你列的每一项他都讲了（折半插入讲得最简，只说「不展开讲，和直接插入一样」，本资料用独立算出的折半查找过程补足了那一步）。**你列了、他讲得更多的：**堆（建立 / 插入 / 删除三个动作逐个演示比较次数）、快速排序（他给的「第二趟 = 左右都做一次」这个定义，以及枢轴方向不能反）。**你没列、他讲了不少的：**① **堆的应用**（今年大纲新增，第七题，含两种思路与复杂度对比）；② **外部排序整块**；③ **学排序的六个角度**这套方法论；④ **哪些排序必须用数组**；⑤ 三路归并与有序段合并的双指针通法。

整合自 3 小时 28 分 44 秒网课讲稿（本地 Whisper 转写，全程无板书；所有序列、堆、归并段均按口述复原）· 未上传任何材料到第三方服务 · 全部例题、序列与次数均回对讲稿原文并独立重算：**17 项过程性推算全部与老师口述一致**（未发现算错），查出**老师口误 1 处**（折半插入「区别在移动次数」，已按正确口径呈现）与**转写硬伤 6 处**（建堆中间结果漏位、小根堆层序崩坏、归并段三处数字串崩坏、三路归并例子不可辨读），已在 §附逐条标注